

NAME:PAILI VARSHITHA

ROLL.NO:2403A510G8

LAB-12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

TASK1:

Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

- Instructions:

- o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.
- o Ask AI to include time complexity and space complexity in the function docstring.
- o Verify the generated code with test cases.

PROMPT:

Write a Python program with a function `merge_sort(arr)` that sorts a list in ascending order, including a docstring with time and space complexity.

Add test cases (empty, single element, sorted, reverse, random) to verify correctness and print results.

CODE:

assignment 12.1.py > merge_sort

```
def merge_sort(arr):  
    if len(arr) <= 1:  
        return arr  
    mid = len(arr) // 2  
    left = merge_sort(arr[:mid])  
    right = merge_sort(arr[mid:])  
    return merge(left, right)  
  
def merge(left, right):  
    result = []  
    i = j = 0  
    while i < len(left) and j < len(right):  
        if left[i] <= right[j]:  
            result.append(left[i])  
            i += 1  
        else:  
            result.append(right[j])  
            j += 1  
    result.extend(left[i:])  
    result.extend(right[j:])  
    return result  
  
test_cases = [  
    [],  
    [42],  
    [1, 2, 3, 4, 5],  
    [5, 4, 3, 2, 1],  
    [3, 1, 4, 1, 5, 9, 2, 6],
```

```
    [5, 4, 3, 2, 1],  
    [3, 1, 4, 1, 5, 9, 2, 6],  
]  
  
for idx, case in enumerate(test_cases, 1):  
    sorted_case = merge_sort(case)  
    print(f"Test case {idx}: Input: {case} => Sorted: {sorted_case}")
```

OUTPUT:

```
ech\2-1\wt\assignment 12.1.py'
Test case 1: Input: [] => Sorted: []
Test case 2: Input: [42] => Sorted: [42]
Test case 3: Input: [1, 2, 3, 4, 5] => Sorted: [1, 2, 3, 4, 5]
Test case 4: Input: [5, 4, 3, 2, 1] => Sorted: [1, 2, 3, 4, 5]
Test case 5: Input: [3, 1, 4, 1, 5, 9, 2, 6] => Sorted: [1, 1, 2, 3, 4, 5, 6, 9]
PS C:\Users\RITHIKA\OneDrive\Desktop\b-tech\2-1\wt> █
```

OBSERVATION:

1. **Correctness** – The function `merge_sort(arr)` successfully sorts lists in ascending order for all tested cases (empty list, single element, already sorted, reverse order, random list).
2. **Stability** – Merge Sort is a stable sorting algorithm, meaning it preserves the order of equal elements.
3. **Efficiency** – The algorithm consistently runs in **$O(n \log n)$** time complexity, which is faster than simple sorts like Bubble or Insertion Sort for large inputs.
4. **Memory Usage** – Merge Sort requires **$O(n)$** extra space because it creates temporary sublists during merging.
5. **Divide and Conquer** – The function works by recursively dividing the array into halves, sorting them, and then merging back, which demonstrates the divide-and-conquer approach.

TASK2:

Task: Use AI to create a binary search function that finds a target element in a sorted list.

- Instructions:

- o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.
- o Include docstrings explaining best, average, and worst-case complexities.
- o Test with various inputs.

PROMPT:

Write a Python program with a function `binary_search(arr, target)` that returns the index of the target in a sorted list or -1 if not found, with a docstring explaining best, average, and worst-case complexities. Add test cases with different inputs (found, not found, first element, last element, empty list) and print results.

CODE:

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

# Test cases
test_cases = [
    ([1, 2, 3, 4, 5], 3),
    ([1, 2, 3, 4, 5], 1),
    ([1, 2, 3, 4, 5], 5),
    ([1, 2, 3, 4, 5], 6),
    ([], 1),
]

for idx, (arr, target) in enumerate(test_cases, 1):
    result = binary_search(arr, target)
    print(f"Test case {idx}: Array: {arr}, Target: {target} => Index: {result}")
```

OUTPUT:

```
ncher' '56281' '--' 'C:\Users\RITHIKA\OneDrive\Desktop\b-tech\2-1\wt\12.2.PY'
Test case 1: Array: [1, 2, 3, 4, 5], Target: 3 => Index: 2
Test case 2: Array: [1, 2, 3, 4, 5], Target: 1 => Index: 0
Test case 3: Array: [1, 2, 3, 4, 5], Target: 5 => Index: 4
Test case 4: Array: [1, 2, 3, 4, 5], Target: 6 => Index: -1
Test case 5: Array: [], Target: 1 => Index: -1
PS C:\Users\RITHIKA\OneDrive\Desktop\b-tech\2-1\wt> ^C
```

OBSERVATION:

1. **Correctness** – The `binary_search(arr, target)` function correctly returns the index of the target element when present and -1 when not found.

2. **Efficiency** – Binary Search is highly efficient compared to linear search, with:

- **Best case:** $O(1)$ (target found at the middle)
- **Average case:** $O(\log n)$
- **Worst case:** $O(\log n)$ (target not present or found at the end of search).

3. **Input requirement** – The list **must be sorted**; otherwise, results will be incorrect.

4.**Edge cases** – Works properly for empty lists, single-element lists, first/last element searches, and cases where the target is absent.

5.**Scalability** – Handles large sorted lists efficiently due to logarithmic complexity.

TASK3:

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to: 1. Quickly search for a product by ID or name. 2. Sort products by price or quantity for stock analysis. • Task: o Use AI to suggest the most efficient search and sort algorithms for this use case. o Implement the recommended algorithms in Python. o Justify the choice based on dataset size, update frequency

PROMT:

Suggest the most efficient search and sort algorithms for a retail inventory system with thousands of products (search by ID/name, sort by price/quantity), justify the choices, and implement them in Python. Include reasoning based on dataset size, update frequency, and performance requirements, and add test cases to verify correctness.

CODE:

```

# Sample inventory: list of dicts
inventory = [
    {'id': 'P001', 'name': 'Apple', 'price': 30, 'quantity': 100},
    {'id': 'P002', 'name': 'Banana', 'price': 10, 'quantity': 150},
    {'id': 'P003', 'name': 'Orange', 'price': 20, 'quantity': 120},
    {'id': 'P004', 'name': 'Grapes', 'price': 40, 'quantity': 80},
    {'id': 'P005', 'name': 'Mango', 'price': 50, 'quantity': 60},
]

# Search by ID (O(1))
def build_id_index(inventory):
    return {item['id']: item for item in inventory}

def search_by_id(id_index, product_id):
    return id_index.get(product_id, None)

# Search by Name (O(log n)), assumes inventory sorted by name
def binary_search_by_name(sorted_inventory, target_name):
    left, right = 0, len(sorted_inventory) - 1
    while left <= right:
        mid = (left + right) // 2
        name = sorted_inventory[mid]['name']
        if name == target_name:
            return sorted_inventory[mid]
        elif name < target_name:
            left = mid + 1
        else:
            right = mid - 1

```

```

        else:
            right = mid - 1
    return None

# Sort by price or quantity (O(n log n))
def sort_inventory(inventory, key):
    return sorted(inventory, key=lambda x: x[key])

# --- Test cases ---
id_index = build_id_index(inventory)

# Search by ID
print("Search by ID 'P003':", search_by_id(id_index, 'P003'))

# Search by Name
inventory_by_name = sort_inventory(inventory, 'name')
print("Search by Name 'Banana':", binary_search_by_name(inventory_by_name, 'Banana'))

# Sort by Price
sorted_by_price = sort_inventory(inventory, 'price')
print("Sorted by Price:", sorted_by_price)

# Sort by Quantity
sorted_by_quantity = sort_inventory(inventory, 'quantity')
print("Sorted by Quantity:", sorted_by_quantity)

```

OUTPUT:

```

ncher' 61355' '---' 'C:\Users\RITHIKA\OneDrive\Desktop\b-tech\2-1\wt\.vscode\12.3.py'
Search by ID 'P003': {'id': 'P003', 'name': 'Orange', 'price': 20, 'quantity': 120}
Search by Name 'Banana': {'id': 'P002', 'name': 'Banana', 'price': 10, 'quantity': 150}
Sorted by Price: [{'id': 'P002', 'name': 'Banana', 'price': 10, 'quantity': 150}, {'id': 'P003', 'name': 'Orange', 'price': 20, 'quantity': 120}, {'id': 'P001', 'name': 'Apple', 'price': 30, 'quantity': 100}, {'id': 'P004', 'name': 'Grapes', 'price': 40, 'quantity': 80}, {'id': 'P005', 'name': 'Mango', 'price': 50, 'quantity': 60}]
Sorted by Quantity: [{'id': 'P005', 'name': 'Mango', 'price': 50, 'quantity': 60}, {'id': 'P004', 'name': 'Grapes', 'price': 40, 'quantity': 80}, {'id': 'P001', 'name': 'Apple', 'price': 30, 'quantity': 100}, {'id': 'P003', 'name': 'Orange', 'price': 20, 'quantity': 120}, {'id': 'P002', 'name': 'Banana', 'price': 10, 'quantity': 150}]
PS C:\Users\RITHIKA\OneDrive\Desktop\b-tech\2-1\wt>

```

OBSERVATION:

- 1.Search** – For ID lookup, a **Hash Map (Dictionary in Python)** provides $O(1)$ average time; for **name search**, Binary Search ($O(\log n)$) is effective if names are stored in sorted order.
- 2.Sort – Timsort** (Python's built-in `sorted()`) is chosen since it handles real-world datasets efficiently with $O(n \log n)$ complexity and optimizations for partially sorted data.
- 3.Performance** – With thousands of products, both searching and sorting run very efficiently; binary search and Timsort scale well without excessive memory usage.
- 4.Update frequency** – Dictionaries allow fast insertions/updates ($O(1)$), while sorting can be done on demand when analysis is needed, ensuring balance between update speed and query efficiency.

5. **Scalability** – The approach handles large datasets smoothly and aligns with retail store requirements for quick lookup and periodic stock analysis.